Pesquisa, ordenação, expressões lambda.

Preparação (antes da aula)

- Executa passo-a-passo este exemplo de procura, tentando sempre prever o resultado.
- Executa passo-a-passo e tenta prever os resultados destes exemplos de ordenação: sorting1 e sorting2.

Exercícios

1. Pretendemos ler o ficheiro `names.txt` (idêntico ao da aula08), mas queremos guardar apenas o apelido e nome próprio de cada pessoa numa lista, no formato "SILVA, NUNO". Complete a função `loadNames` no programa `names9.py` para fazer isso e teste o programa.

   ```
   $ python3 names9.py
   ['ABRANTES, PEDRO', 'ABREU, ANDREIA', 'ABREU, CARLOS', 'AGUIAR, ANA']
   ```

   A seguir o programa pede um apelido e chama a função `findSurname` para mostrar a lista de todos os nomes com esse apelido (e os nomes vizinhos). Complete e corrija as chamadas às funções `bisect_left` e `bissect_right` para conseguir isso.

   ```
   Surname? (ENTER to stop) Sousa
   Searching 'SOUSA' --> [254:256]
   names[253:254] =  ['SOARES, VITOR']
   names[254:256] =  ['SOUSA, ALVARO', 'SOUSA, MARTA']
   names[256:257] =  ['TAVARES, LUIS']
   ```

   Experimente procurar os apelidos: SANTOS, SA (sem acento) e ESTRANHO. Confirme que funciona bem.

   Comente a instrução `names.sort()` na função principal e teste de novo. Funciona?

   E se fizer uma ordenação `names.sort(key=len)`, funciona?

2. Na aula07 fez um programa que conta ocorrências de letras num ficheiro de texto. Crie uma nova versão desse programa que liste o resultado por ordem decrescente do número de ocorrências.

   Sugestão: Use o método `sorted` com os argumentos `key=` e `reverse=` para ordenar a sequência de pares chave-valor (`items`) do dicionário.

   ```
   $ python3 countLetters2.py wordlist.txt
   e 61167
   s 47192
   i 45719
   ```

3. O programa `tabelaFutebol.py` tem uma lista com a tabela classificativa de um campeonato de futebol. Cada elemento da lista é um tuplo com o nome da equipa e números de vitórias, de empates, de derrotas, de golos marcados e sofridos. O programa já tem uma função para mostrar a tabela devidamente formatada e uma função, definida

por uma expressão lambda, para determinar o número de jogos realizados por uma equipa. Complete o programa nos locais indicados para resolver cada uma das alíneas.

a) Complete a expressão lambda para definir a função `pontos` que, dado um registo de uma equipa, deve devolver o número de pontos da equipa. (Cada vitória vale 3 pontos, cada empate vale 1 ponto.)

b) Acrescente os argumentos adequados à função `sorted` para obter uma tabela ordenada por ordem decrescente de pontos.

c) Acrescente os argumentos adequados à função `sorted` para obter uma tabela ordenada por ordem decrescente da diferença de golos marcados e sofridos.

d) Acrescente os argumentos adequados à função `sorted` para ordenar a tabela por ordem decrescente de pontos e, se iguais, pela diferença de golos.

4. Faça uma função que calcule a mediana de uma lista de valores. Se a lista tiver um número ímpar de valores, a mediana é o valor no meio da lista ordenada. Se a lista tiver um número par de valores, a mediana é a média dos dois valores no meio da lista ordenada. No entanto, repare que a função deve preservar o conteúdo da lista intacto. (Pode testar no CodeCheck.)

5. O programa `multsort.py` lê uma lista de notas de vários alunos do ficheiro `grades.json` para `lst1` e mostra-a em forma de tabela. Cada elemento da lista é também uma lista com [Número, Nota, Nome, Grupo]. Complete o programa para ordenar a lista por ordem decrescente da nota (em `lst2`). Depois ordene o resultado por ordem crescente do grupo (em `lst3`).

   Repare que o resultado final fica ordenado por grupo, mas dentro de cada grupo, mantém a ordem anterior (por nota decrescente). Isso é garantido porque a linguagem Python usa um algoritmo de ordenação estável. Desta forma, podemos usar ordenações sucessivas para obter ordenações por critérios compostos.

6. *O ficheiro `wordlist.txt` contém uma lista de palavras de língua inglesa, por ordem. Leia essas palavras para uma lista e, usando uma função de pesquisa binária (do módulo `bisect`), descubra quantas palavras começam por "ea", sem ter de percorrer tudo. *Sugestão: procure a primeira palavra com "ea" e a primeira com "eb" e subtraia os índices*. E quantas palavras começam por "truo"? Nenhuma? Então qual é a primeira letra, maior do que 'o', que ocorre após um "tru", nas palavras inglesas?

7. *Usando o mesmo princípio, faça uma função que indique todas as letras que podem suceder a um certo prefixo. Pode usar esta função num sistema de escrita inteligente que vai apresentando as letras possíveis para completar um certo prefixo já introduzido. Quando o utilizador introduz mais uma letra, é acrescentada ao prefixo anterior e apresenta-se nova lista de possibilidades e assim sucessivamente.

8. *O programa `insertionSort.py` tem uma implementação do algoritmo de ordenação por inserção. Modifique a função para aceitar um argumento opcional `key=` que funcione tal como na função pré-definida `sorted`.

9. *No programa `polynomial.py` pretendemos definir uma função que crie um polinómio de segundo grau arbitrário. Ou seja, `p = polynomial2(2, -1, 3)`

deve colocar em `p` uma <u>função</u> tal que `p(x)` calcule o valor $p(x) = 2x^2 - x + 3$ para qualquer valor `x` que lhe seja passado. Note que dentro do corpo de `polynomial2` terá de definir uma nova função para devolver. Essa definição poderá ser feita com uma instrução `def` ou com uma expressão `lambda`.

10. Integração numérica de uma função pela regra dos trapézios. (CodeCheck integrate1)